

SIMATS ENGINEERING

ASSESSMENT TOOL 1

COURSE NAME: INTERNET PROGRAMMING
COURSE CODE: CSA43

COURSE OUTCOME COVERED

CO-2 : Apply CSS layout and styling techniques to create visually appealing and responsive web interfaces, and use JavaScript to enable interactive client-side behavior. (L3)

Assessment Tool : Integrated Experiments

Weightage: 40%

QUESTIONS: Total Marks: 30

Experiment 1: Responsive Layout Design

Suppose that a retail website is not responsive across mobile and tablet devices, causing misalignment of content and improper spacing.

(a) Analyze the possible reasons for lack of responsiveness in the existing layout. (b) Design a responsive webpage layout using **CSS Flexbox or Grid**, ensuring proper alignment of header, navigation, content, and footer.

(c) Demonstrate how the layout adapts to **different screen sizes (mobile, tablet, desktop)** using media queries.

(d) Implement spacing, alignment, and distribution techniques to improve visual consistency. (e) Evaluate the effectiveness of the responsive design and suggest further improvements.

Answer:

(a) Reasons for lack of responsiveness: The site probably fixes the outer wrapper to a single desktop pixel width, positions the navigation and sidebar with float:left/float:right (which does not reflow gracefully), sizes images and buttons in fixed px rather than % or rem, and ships no viewport meta tag, so mobile browsers shrink the whole desktop layout instead of re-laying it out.

(b) Responsive layout using CSS Flexbox, mobile-first: The base styles target small screens and stack everything vertically; a single media query then switches to a row layout for larger screens, rather than starting from a fixed desktop design and squeezing it down.

<body>

```

<header class="header">Header</header>
<div class="body-wrap">
  <nav class="nav">Nav</nav>
  <main class="content">Content</main>
</div>
<footer class="footer">Footer</footer>
</body>
/* Mobile-first base styles */
.body-wrap { display: flex; flex-direction: column; }
.nav, .content { width: 100%; }

@media (min-width: 768px) {
  .body-wrap { flex-direction: row; }
  .nav { flex: 0 0 220px; }
  .content { flex: 1 1 auto; }
}

```

(c) Adapting to screen sizes: Three breakpoints are used - up to 767px (phones) the nav sits above the content as a horizontal bar; 768px-1023px (tablets) the nav becomes a fixed 220px sidebar with a narrower content column; 1024px+ (desktop) the same sidebar layout gets extra max-width and centered margins so lines of text don't stretch too wide.

```

@media (min-width: 768px) and (max-width: 1023px) {
  .content { padding: 1rem; }
}
@media (min-width: 1024px) {
  body { max-width: 1200px; margin: 0 auto; }
  .content { padding: 2rem; }
}

```

(d) Spacing, alignment and distribution: Instead of hard-coded margins between elements, flex's gap property is used for even spacing, and justify-content/align-items handle alignment so it doesn't need to be recalculated at each breakpoint.

```

.body-wrap { gap: 1.5rem; }
.header, .footer {
  display: flex;
  align-items: center;
  justify-content: space-between;
  padding: 1rem 1.5rem;
}

```

(e) Evaluation and improvements: Resizing the browser from 320px up to 1440px shows a clean single reflow point at 768px with no horizontal scrollbars or clipped text. A limitation of this version is that the two breakpoints are fairly coarse; adding an intermediate breakpoint around 480px for larger phones, using clamp() for font sizes instead of fixed rem jumps, and auditing tap-target sizes (minimum ~44px) would further improve the experience on real devices.

Experiment 2: Form Validation using JavaScript

Suppose that a registration form accepts invalid email and phone number inputs, leading to incorrect data submission.

(a) Analyze the issues in the existing form validation mechanism.

(b) Design input fields for email and phone number with appropriate HTML attributes. (c) Implement **JavaScript validation using Regular Expressions** for both email and phone number.

(d) Develop a mechanism to display **real-time error messages** for invalid inputs. (e) Evaluate the validation system and suggest enhancements for better user experience and security.

Answer:

(a) Issues in the existing validation: Beyond weak pattern checking, the form likely gives no distinction between 'empty' and 'invalid' states, shows no error until the whole form is submitted (a poor experience on long forms), and does not disable the submit button while

invalid, so users can trigger repeated failed submissions.

(b) Input fields with appropriate HTML attributes: This version leans on the built-in Constraint Validation API (pattern, required) so the browser and the JavaScript share one source of truth for what counts as valid.

```
<form id="regForm" novalidate>
  <input type="email" id="email" required
    pattern="[A-Za-z0-9._%+-]+@[A-Za-z0-9.-]+\.[A-Za-z]{2,}"
    title="Enter a valid email address">
  <span class="error" id="emailError"></span>

  <input type="tel" id="phone" required
    pattern="[6-9][0-9]{9}"
    title="Enter a valid 10-digit phone number">
  <span class="error" id="phoneError"></span>

  <button type="submit" id="submitBtn" disabled>Register</button>
</form>
```

(c) JavaScript validation with a single reusable validator: Rather than one bespoke listener per field, a small `validateField()` helper reads each input's own pattern attribute, so adding a new field later needs no extra JavaScript.

```
function validateField(input, errorEl, message) {
  const ok = input.checkValidity();
  errorEl.textContent = ok ? '' : message;
  input.classList.toggle('invalid', !ok);
  return ok;
}
```

```
const fields = [
  { input: document.getElementById('email'), error: document.getElementById('emailError'),
    message: 'Enter a valid email address' },
  { input: document.getElementById('phone'), error: document.getElementById('phoneError'),
    message: 'Enter a valid 10-digit phone number' }
];
```

(d) Real-time error messages and submit control: The submit button stays disabled until every field passes, and each field re-checks itself as the user types.

```
const submitBtn = document.getElementById('submitBtn');

function refreshForm() {
  const results = fields.map(f => validateField(f.input, f.error, f.message));
  submitBtn.disabled = results.includes(false);
}
```

```
fields.forEach(f => f.input.addEventListener('input', refreshForm));
document.getElementById('regForm').addEventListener('submit', e => {
  e.preventDefault();
  refreshForm();
  if (!submitBtn.disabled) { /* proceed with submission */ }
});
```

(e) Evaluation and enhancements: Tying validation to the native pattern attribute keeps client and browser rules consistent and disabling the submit button prevents obviously-invalid attempts, but client-side checks can still be bypassed by disabling JavaScript or calling the endpoint directly, so the server must re-run the same regex checks. Further enhancements: normalize phone numbers (strip spaces/dashes) before testing, add a honeypot field or CAPTCHA against bots, and log repeated validation failures for abuse monitoring.

Experiment 3: CSS Layout Debugging

Suppose that a webpage layout overlaps when the browser window is resized, affecting usability.

(a) Analyze the CSS properties responsible for layout overlapping issues. (b) Examine the role of the **CSS box model (margin, border, padding, content)** in layout design.

(c) Debug the layout using appropriate **positioning techniques (relative, absolute, flex, grid)**.

(d) Modify the CSS to ensure proper alignment and prevent overlapping during resizing. (e) Evaluate the corrected layout and suggest best practices for maintaining consistency.

Answer:

(a) CSS properties responsible for overlap: In this scenario the likely causes are elements taken out of normal flow with `position: absolute` but sharing the same offsets, sibling elements with conflicting z-index values inside different stacking contexts (so a higher z-index doesn't 'win' because its ancestor has a lower one), and fixed widths on flex/inline-block children that don't shrink when the container narrows, forcing them to sit on top of each other.

(b) Role of the CSS box model: Because total rendered width = content + padding + border (+ margin for spacing between boxes), two elements each set to 50% width with additional padding under content-box sizing can together exceed 100% of the parent, causing the second one to wrap underneath and visually overlap the first during a resize. Switching to box-sizing: border-box makes the declared width include padding and border, keeping the math predictable.

(c) Debugging with positioning techniques: The fix here treats the columns as a CSS Grid instead of two independently offset absolute boxes, so the browser handles collision-free placement automatically; `position: relative/absolute` is reserved only for a badge that must sit visually on top of one card.

(d) Corrected CSS:

```
*, *::before, *::after { box-sizing: border-box; }

.grid {
  display: grid;
  grid-template-columns: repeat(auto-fit, minmax(240px, 1fr));
  gap: 1rem;
}
.card { position: relative; padding: 1rem; }
.badge {
  position: absolute;
  top: -0.5rem;
  right: -0.5rem;
  z-index: 1;
}
```

(e) Evaluation and best practices: With auto-fit/minmax the grid recalculates how many columns fit at the current width and never forces two cards into the same space, and the badge stays correctly pinned to its own card because its stacking context is scoped to that card's `position: relative`. Best practices: apply a universal border-box reset early in the stylesheet, prefer Grid's auto-fit/minmax over manually calculated percentage widths, keep z-index usage local and documented (e.g. a small scale like 1, 10, 100) rather than arbitrary large numbers, and verify at several widths using dev-tools rather than only the designer's original screen size.

Code of Conduct:

I (K.sai kumar / 192372080) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:

K.sai kumar